

Dear Diary

⚠ **Security Warning:** Work in an isolated environment (virtual machine or container). Never open unknown files on your primary machine.

Challenge Description

If you can find the flag on this disk image, we can close the case for good!

Hints

- If you're observing binary data raw in the terminal you may be misled about the contents of a block.
-

Tools Used

- `7z` – Extract compressed files
 - `file` – Determine file type
 - `strings` – Extract printable character sequences
 - `exiftool` – Read metadata
 - `fdisk` – View partition tables
 - `fls` – List files in a disk image (Sleuth Kit)
 - `icat` – Retrieve a file by inode number (Sleuth Kit)
 - `xxd` – View hexadecimal representation
 - **Autopsy** – GUI forensic analysis tool
-

Complete Solution

Step 1: Download and Extract

Download the disk image named `disk.flag.img.gz` from the challenge page.

Extract the compressed disk image:

```
7z x disk.flag.img.gz
```

This decompresses the gzipped file and creates the raw disk image `disk.flag.img`.

Step 2: Basic File Inspection

First, verify the file type:

```
file disk.flag.img
```

Expected output:

```
disk.flag.img: DOS/MBR boot sector; partition 1 : ID=0x83, active, start-CHS  
(0x0,32,33), end-CHS (0x26,94,56), startsector 2048, 614400 sectors; partition 2 :  
ID=0x82, start-CHS (0x26,94,57), end-CHS (0x47,1,58), startsector 616448, 524288  
sectors; partition 3 : ID=0x83, start-CHS (0x47,1,59), end-CHS (0x82,138,8),  
startsector 1140736, 956416 sectors
```

The output confirms this is a **DOS/MBR boot sector** with three partitions. The third partition (ID `0x83`) is our target – it's a Linux filesystem.

Step 3: Search for Visible Text

Check if the flag appears as plain text:

```
strings disk.flag.img | grep -i "picoCTF"
```

Output: (no results)

The flag isn't directly visible. We need to dig deeper.

Step 4: Inspect Metadata

```
exiftool disk.flag.img
```

Output:

```
ExifTool Version Number      : 13.50
File Name                    : disk.flag.img
Directory                    : .
File Size                    : 1074 MB
...
Error                        : Unknown file type
```

Nothing suspicious.

Step 5: View Partitions with fdisk

```
fdisk -l disk.flag.img
```

Output:

```
Disk disk.flag.img: 1 GiB, 1073741824 bytes, 2097152 sectors
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disklabel type: dos
Disk identifier: 0x6062d30a
```

Device	Boot	Start	End	Sectors	Size	Id	Type
disk.flag.img1	*	2048	616447	614400	300M	83	Linux
disk.flag.img2		616448	1140735	524288	256M	82	Linux swap / Solaris
disk.flag.img3		1140736	2097151	956416	467M	83	Linux

The third partition starts at sector [1140736](#) . This is where the user data lives.

Step 6: List Files with fls (Sleuth Kit)

Use `fls` to list all files and directories in the third partition:

```
fls -o 1140736 disk.flag.img
```

Output:

```
d/d 32513:      home
d/d 11:  lost+found
d/d 32385:      boot
d/d 64769:      etc
d/d 32386:      proc
d/d 13:  dev
d/d 32387:      tmp
d/d 14:  lib
d/d 32388:      var
d/d 21:  usr
d/d 32393:      bin
d/d 32395:      sbin
d/d 32539:      media
d/d 203:      mnt
d/d 32543:      opt
d/d 204:      root
d/d 32544:      run
d/d 205:      srv
d/d 32545:      sys
d/d 32530:      swap
V/V 119417:     $OrphanFiles
```

The `root` directory (inode `204`) looks promising. Let's explore it recursively:

```
fls -r -p -o 1140736 disk.flag.img 204
```

Output:

```
r/r 1837:      .ash_history
d/d 1842:      secret-secrets
r/r 1843:      secret-secrets/force-wait.sh
r/r 1844:      secret-secrets/innocuous-file.txt
r/r 1845:      secret-secrets/its-all-in-the-name
```

Key finding: A directory named `secret-secrets` containing several files. The file `innocuous-file.txt` (inode `1844`) sounds suspicious – it claims to be innocuous, but that's likely a deception!

Step 7: Extract the Innocuous File

```
icat -o 1140736 disk.flag.img 1844
```

Output: (nothing)

The file appears empty. However, the hint suggests that binary data viewed in a terminal can be misleading. The file may contain hidden data or the flag may be split across multiple locations.

Step 8: Examine the Directory Metadata

Let's examine the `secret-secrets` directory itself:

```
icat -o 1140736 disk.flag.img 1842 | xxd | grep ".txt" -A3
```

Output:

```
00000040: 732d 6669 6c65 2e74 7874 0000 0000 0000  s-file.txt.....
00000050: 0000 0000 0000 0000 3507 0000 9c03 1301  .....5.....
00000060: 6974 732d 616c 6c2d 696e 2d74 6865 2d6e  its-all-in-the-n
00000070: 616d 6500 0000 0000 0000 0000 0000 0000  ame.....
```

Interesting! The directory entry references `its-all-in-the-name`. The flag might be hiding in the file names or slack space.

Step 9: Locate String Offsets

Find where `innocuous-file.txt` appears in the raw disk image:

```
strings -t x disk.flag.img | grep "innocuous-file.txt"
```

Output:

```
23c3c838 innocuous-file.txt
2b6f8c38 innocuous-file.txt
2b6fc038 innocuous-file.txt
2b6fe038 innocuous-file.txt
```

```
2b6ff838 innocuous-file.txt
2b701c38 innocuous-file.txt
2b704038 innocuous-file.txt
2b706438 innocuous-file.txt
2b707c38 innocuous-file.txt
2b70a038 innocuous-file.txt
2b70b838 innocuous-file.txt
2b70dc38 innocuous-file.txt
2b710038 innocuous-file.txt
2b711838 innocuous-file.txt
```

Multiple offsets – the filename appears many times, likely in directory entries and slack space.

Step 10: Examine Each Offset

Let's examine each offset to find flag fragments:

```
xxd -s 0x23c3c838 -l 64 disk.flag.img
```

Output:

```
23c3c838: 696e 6e6f 6375 6f75 732d 6669 6c65 2e74  innocuous-file.t
23c3c848: 7874 0000 0000 0000 0000 0000 0000 0000  xt.....
23c3c858: 3507 0000 9c03 1301 6974 732d 616c 6c2d  5.....its-all-
23c3c868: 696e 2d74 6865 2d6e 616d 6500 0000 0000  in-the-name.....
```

This shows the directory entry with the name `its-all-in-the-name`.

Now examine `0x2b6fc038`:

```
xxd -s 0x2b6fc038 -l 64 disk.flag.img
```

Output:

```
2b6fc038: 696e 6e6f 6375 6f75 732d 6669 6c65 2e74  innocuous-file.t
2b6fc048: 7874 0000 3507 0000 a803 1101 6f72 6967  xt..5.....orig
2b6fc058: 696e 616c 2d66 696c 656e 616d 6500 0000  inal-filename...
2b6fc068: 0000 0000 0000 0000 0000 0000 0000 0000  .....
```

Now examine `0x2b6fe038`:

```
xxd -s 0x2b6fe038 -l 64 disk.flag.img
```

Output:

```
2b6fe038: 696e 6e6f 6375 6f75 732d 6669 6c65 2e74  innocuous-file.t
2b6fe048: 7874 0000 0000 0000 0000 0000 0000 0000  xt.....
2b6fe058: 0000 0000 0000 0000 0000 0000 0000 0000  .....
2b6fe068: 3507 0000 8c03 0301 7069 6300 0000 0000  5.....pic.....
```

Here we see 7069 63 which is ASCII for pic ! This is the beginning of the flag!

Step 11: Collect Flag Fragments

Continue examining each offset:

Offset	Fragment Found
0x2b6fe068	pic
0x2b6ff838	oCT
0x2b701c38	F{1
0x2b704038	_53
0x2b706438	3_n
0x2b707c38	4m3
0x2b70a038	5_8
0x2b70b838	0d2
0x2b70dc38	4b3
0x2b710038	0}

Step 12: Assemble the Flag

Combine all fragments:

```
picoCTF{<REDACTED>}
```

Step 13: Alternative Method – Using Autopsy (GUI)

If you prefer a graphical approach, use **Autopsy** (the Sleuth Kit GUI):

- 1. **Launch Autopsy** and create a new case
- 2. **Add a host** and import `disk.flag.img` as an image file
- 3. **Run analysis** and select the **Keyword Search** module
- 4. **Search for** `.txt` to find all text file references
- 5. **Review the results** – each result contains a fragment of the flag
- 6. **Combine the fragments** to reveal the complete flag

 Final Result

```
picoCTF{<REDACTED>}
```

Note: The actual flag may vary. In the real challenge, assembling the fragments from the offsets will reveal the complete flag.

 Key Concepts

Concept	Description
Slack Space	Unused bytes at the end of a disk block that can contain residual data.
Directory Entry Fragments	Deleted or overwritten directory entries may still contain filename fragments.

Concept	Description
Data Carving	Extracting hidden data by examining raw disk offsets.
Sleuth Kit (TSK)	A collection of forensic tools for analyzing disk images (<code>fls</code> , <code>icat</code> , <code>mmls</code>).
Autopsy	The GUI front-end for The Sleuth Kit.
Inode	A data structure that stores metadata about a file (permissions, timestamps, pointers to data blocks).

Pro Tips

1. **Don't trust file names** – `innocuous-file.txt` is clearly trying to hide something.
 2. **Raw terminal output can be misleading** – Use `xxd` or a hex viewer to see the actual bytes.
 3. **Flag fragments can be in slack space** – Examine the raw disk at offsets where filenames appear.
 4. `strings -t x` **is invaluable** – It shows the offset of every printable string, helping you locate hidden data.
 5. **Use `xxd -s` to jump to specific offsets** – This lets you examine raw bytes at any location.
 6. **Autopsy makes it easier** – If you're overwhelmed by command-line tools, try the GUI for visual analysis.
-

Alternative Tools

Tool	Purpose
Autopsy	GUI forensic analysis platform
Sleuth Kit (<code>fls</code> , <code>icat</code>)	Command-line forensic tools
<code>xxd</code>	Hex dump utility

Tool	Purpose
HxD (Windows)	Hex editor
FTK Imager	Disk imaging and analysis